

如何设计一个好用的 Agent 系统

系统调研报告（初稿）

更新日期：2026-04-28

一句话结论

“好用”的 Agent 系统，重点不在把模型塑造成高度自治体，而在把任务边界、工具接口、状态恢复、评测闭环做好。Agent + 显式工作流比仓促上多 Agent 更容易真正落地。

摘要

本报告围绕“如何设计一个好用的 Agent 系统”进行一轮系统调研，综合官方文档、代表性论文、评测基准与开源项目，回答三个问题：第一，Agent 系统的主流理解和设计重点是什么；第二，哪些能力是从 demo 走向生产时必须补齐的；第三，在技术选型上有哪些趋势。

综合 2023 至 2026 年公开资料，本报告认为 Agent 系统设计正在从“让模型自己多想一点”转向“让系统在可控框架下工作”。核心要素包括：workflow、tool use、state、checkpoint、human review、guardrails 和 evals。Agent 的价值来自可执行工作流，而非单纯调用 API。

1. 核心结论

- 先把单 Agent、工具调用和显式工作流做好，再考虑多 Agent。多 Agent 只有在并行拆分、权限分离、角色分工明确时才有意义。
- Agent-Computer Interface (ACI) 是成败关键。工具描述、参数 schema、执行反馈、错误码、返回格式，缺一不可。
- 好用的 Agent 必须支持状态持久化与恢复。长任务、外部依赖、人工审批、失败重试都要求 checkpoint、thread/session 和 resume 能力。
- 评测必须从第一天开始建设。静态 benchmark 只能提供参考，生产可用性仍取决于贴近业务任务的在线或离线测试。
- 安全设计必须内建，而不是上线前补丁。尤其要关注 prompt injection、工具滥用、权限越权、记忆污染和高频幻觉。

2. 什么叫“好用”的 Agent 系统

从用户视角看，好用意味着结果稳定、失败可解释、上下文衔接自然、必要时可让人接管；从工程视角看，好用意味着可观测、可调试、可灰度、可替换组件、可回滚。Agent 系统的 tradeoff 是可管理的。

视角	“好用”的表现	系统设计含义
用户	少出错、能解释、可继续	需要清晰的任务边界、失败处理和会话状态
工程	能调试、能灰度、能替换组件	需要 tracing、evals、typed tools、可观测性
业务	单位成本可接受、成功率可提升	需要明确 KPI、任务分层和人机协同机制

3. 官方文档传递出的主流设计方向

Anthropic、OpenAI 和 Microsoft 在 2025 至 2026 年的公开文档中传递出一个非常一致的信号：Agent 不是单个模型能力的别名，而是模型、工具、知识、控制流、状态和安全机制的组合系统。

来源	核心观点	对设计的启发
Anthropic	先从简单模式开始，只有在复杂度真i agentic 系统	优先 workflow 设计，不迷信多 Agent
OpenAI	构建 agent 是设计 workflow 并连接模型、工具、guardrails、知识 logic 的过程	把工具、逻辑节点、评测与安全一并纳入系统设计
Microsoft Agent Framework	明确区分 agents 与 workflows，并强调 session、checkpoint、type safety、MCP、telemetry	生产系统更需要强编排与长任务恢复能力

判断

“能不能把流程写成函数” 仍然是一个很好的分界线。若任务可以被稳定规则化处理，优先规则流或传统服务；才更值得上场。

4. 研究与论文脉络

近三年的关键论文大致可以分成五条路线：推理与行动结合、工具使用、规划、反思与记忆、搜索与评测。它们 Agent 系统的主流设计语言。

方向	代表论文	关键启发
Reasoning + Acting	ReAct (2023)	把推理轨迹与环境动作交错执行，降低幻觉并增强可解释性
Tool Use	Toolformer (2023)	关键问题不只是能否调工具，而是何时调、如何传参、如何吸
Planning	Plan-and-Solve (2023)	先规划再执行能显著改善多步任务中的遗漏与失误
Reflection / Memory	Reflexion (2023)	语言反馈与反思记忆可以替代昂贵微调，适合 evaluator-optimizer 结构
Search	LATS (2023)	对高难任务，规划与树搜索能提升成功率，但代价是更高的成

两篇综述尤其值得作为总览入口。《A Survey on Large Language Model based Autonomous Agents》提出了涵盖 profile、memory、planning、action 和 environment feedback 的统一分析框架；《Large Language Model based Multi-Agents: A Survey of Progress and Challenges》则更系统地梳理了多 Agent 系统中的角色设定、通信和协作问题。

5. 设计一个好用 Agent 系统的关键模块

1. 任务建模：先定义任务类型、输入输出、完成标准、允许失败的范围，再决定是否需要 Agent，而不是反过来
2. 工具层与 ACI：工具应具备清晰的名称、用途、参数 schema、返回结构、错误信息和权限说明。模型最怕模
3. 编排层：需要支持 routing、branching、parallelism、retry、budget control 与 stop condition。真正影响稳定性的往往是编排，不是模型本身。

- 4. 状态与记忆：至少区分短期工作记忆、会话状态、长期偏好或知识。并不是所有“记忆”都该长期保留。
- 5. 观察与评测：要能回看每一步输入、工具调用、模型输出、人工干预和最终结果，并把失败归因到具体环节。
- 6. 安全与治理：对写文件、发消息、调用外部系统、执行代码等高风险动作设置审批、白名单或策略检查。

6. 代表性开源项目盘点

项目	定位	优势	初步判断
OpenAI Agents SDK	应用级 Agent 开发工具包	tools、handoffs、guardrail 概念完整	适合自控编排和生产集成
LangGraph	低层图式编排框架	durable execution、human-in-the-loop、memory 很强	适合复杂长流程和强状态系统
Microsoft Agent Framework	企业级 Agent + Workflow 框架	type safety、session、checkpoi	适合企业级长期演进
AutoGen	老牌多 Agent 框架	历史影响大、资料多	已进入 maintenance mode，新项目不宜首选
CrewAI	强调 Crews 与 Flows	概念包装清楚、上手快	适合业务自动化原型，但要警惕过早 Agent 化
smolagents	轻量代码型 Agent 框架	简洁透明、适合实验	适合原型和教学，不必承担过多生产期望
SWE-agent / OpenHands	垂直领域 software agent	ACI 与环境反馈设计很有参考价值	适合学习真实任务闭环，不是通用框架替代

7. 评测与 benchmark 的启发

AgentBench、SWE-bench、OSWorld 等 benchmark 为比较不同系统提供了共同语言，但它们并不能直接替代。2026 年 2 月 23 日，OpenAI 发布《Why we no longer evaluate SWE-bench Verified》，明确指出单一公开基准对 agent 的区分能力正在下降，这个信号很值得注意。

Benchmark / 资料	覆盖能力	对系统设计的启发
AgentBench	通用 agent 任务	适合认识任务多样性，但离生产环境仍有距离
SWE-bench / SWE-agent	代码修复与仓库级任务	强调环境、测试反馈和 ACI 的重要性
OSWorld	真实电脑环境中的多模	说明 GUI grounding、操作知识、长期任务仍是难点
OpenAI 对 SWE-bench Verified 的反思	benchmark 有老化和污染风险	企业必须建立自己的 eval flywheel

8. 初步技术判断

- 最稳妥的系统形态通常是“workflow 外壳+少量专业 agent 节点+强工具接口+强评测闭环”，而不是无限制
- 多 Agent 的价值主要来自任务分治、并行执行、上下文隔离和角色审核，而不是“像组织一样协作”这种叙事

- 对多数企业任务，真正的难点是外部系统集成、权限设计、追踪与审计，而不是 prompt engineering。
- 长期记忆不是默认加分项。若没有生命周期、可信度和失效策略，记忆会迅速从能力变成风险源。
- 未来框架竞争的核心将更多落在状态管理、互操作协议、评测平台和人机协同体验，而不只是 agent DSL。

9. 建议的落地路线

阶段	目标	建议动作
阶段 1：任务选型	识别值得 agent 化的任务	选择高价值、步骤不完全固定、能获得外部反馈的任务
阶段 2：最小系统	验证单 Agent 闭环	实现任务输入、工具调用、停止条件、失败回收和 tracing
阶段 3：生产化底座	提高稳定性	补齐状态持久化、审批流、权限控制、监控和评测集
阶段 4：结构升级	在必要时引入多 Agent	仅在单 Agent 上下文已失控或确实需要角色分工时再拆分

10. 参考资料（节选）

1. Anthropic. Building Effective Agents. <https://www.anthropic.com/engineering/building-effective-agents>
2. OpenAI. Agents. <https://platform.openai.com/docs/guides/agents>
3. OpenAI. Agent evals. <https://platform.openai.com/docs/guides/agent-evals>
4. OpenAI. Safety in building agents. <https://platform.openai.com/docs/guides/agent-builder-safety>
5. Microsoft Learn. Microsoft Agent Framework Overview. <https://learn.microsoft.com/en-us/agent-framework/overview/>（页面显示最后更新：2026-02-20）
6. Model Context Protocol. <https://modelcontextprotocol.io/schema/v1>；<https://modelcontextprotocol.io/specification/2024-11-05/architecture/index>
7. Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. <https://arxiv.org/abs/2210.03629>
8. Schick et al. Toolformer: Language Models Can Teach Themselves to Use Tools. <https://arxiv.org/abs/2302.04761>
9. Wang et al. Plan-and-Solve Prompting. <https://arxiv.org/abs/2305.04091>
10. Shinn et al. Reflexion: Language Agents with Verbal Reinforcement Learning. <https://arxiv.org/abs/2303.11366>
11. Zhou et al. Language Agent Tree Search. <https://arxiv.org/abs/2310.04406>
12. Wang et al. A Survey on Large Language Model based Autonomous Agents. <https://arxiv.org/abs/2308.11432>
13. Guo et al. Large Language Model based Multi-Agents. <https://arxiv.org/abs/2402.01680>
14. Liu et al. AgentBench. <https://arxiv.org/abs/2308.03688>
15. Jimenez et al. SWE-bench. <https://arxiv.org/abs/2310.06770>

16. Xie et al. OSWorld. <https://arxiv.org/abs/2404.07972>
17. OpenAI. Why we no longer evaluate SWE-bench Verified. <https://openai.com/index/why-we-no-longer-evaluate>
18. OpenAI Agents SDK. <https://github.com/openai/openai-agents-python>
19. LangGraph. <https://github.com/langchain-ai/langgraph>
20. Microsoft Agent Framework. <https://github.com/microsoft/agent-framework>
21. AutoGen. <https://github.com/microsoft/autogen>
22. CrewAI. <https://github.com/crewAIInc/crewAI>
23. smolagents. <https://github.com/huggingface/smolagents>
24. OpenHands. <https://github.com/All-Hands-AI/OpenHands>